Development of an AI platform for software quality assurance automation based on large language models

Pre-defense 1 materials · 24 October 2026

Team: Қайдар Нариман Нұрланұлы · Таниберген Бекнұр Болатбекұлы · Сариев Максот Бахтыгалиевич (SE-2406)

Supervisor: Айтмұханбетова Эльвира Айтмұханбетқызы, senior lecturer

Software Engineering · Astana IT University, School of Software Engineering

Repository: https://github.com/KNari06/qait-diploma
YouTrack: https://qait-diploma.youtrack.cloud/projects/QAIT

Contents

Pre-defense 1: Requirements Checklist

Pre-defense 1: Submission Report

1. Project proposal

2. Evidence of the relevance

3. Literature review

4. Existing system comparison

5. Technology comparison

6. Requirements specification

7. Architecture and diagrams

8. UX / prototype

9. Verification and evaluation plan

10. Project management and risks

11. Supervisor feedback

Appendix A. Repository README

Appendix B. YouTrack backlog

Pre-defense 1: Requirements Checklist

How this submission covers the components and the Git repository requirements set for 24 October.

A. Components

№	Component	What the committee evaluates	Where in this submission
1	Project proposal	Whether the problem is clearly defined, the scope is realistic, and the project involves sufficient engineering complexity	Section 1: problem 1.1, users 1.2, aim 1.3, objectives 1.4, outcomes 1.5, scope 1.6, complexity 1.7
2	Evidence of the relevance	What evidence informs the requirements and whether verified findings are distinguished from assumptions	Section 2: assumptions A1–A7, interview guide, proto-personas (hypothetical) 2.5, evidence from literature and existing systems 2.6, interview findings 2.7
3	Literature review	What approaches exist, what is already known, and which limitations the project takes into account	Section 3: 20 sources (18 peer-reviewed, most from 2020–2025, foundational works kept), analysis table 3.2, synthesis 3.3, gaps 3.4
4	Existing system comparison	Why the proposed solution is needed and whether claims about existing systems are supported by evidence	Section 4: 5 systems × 9 criteria 4.3, conclusions 4.4, evidence log with sources and dates 4.5
5	Technology comparison	Whether technology choices are justified by the project requirements	Section 5: 6 key decisions (client, backend, storage, service interaction, browser automation, LLM), 3 alternatives each, weighted criteria linked to requirements
6	Requirements specification	Whether the fulfilment of each mandatory requirement can be verified	Section 6: 30 functional + 11 non-functional requirements, MoSCoW priorities, acceptance criterion for every Must, source of each requirement, traceability 6.4
7	Architecture and diagrams	Whether requirements, components, data, and scenarios are aligned	Section 7: system context 7.1, use case diagram with requirement mapping 7.2, component diagram (draft) with owners 7.3, data model 7.4, two sequence diagrams 7.5, architectural decisions 7.6; editable PlantUML sources in `diagrams/src`
8	UX / prototype	Whether it is clear how users or external systems will interact with the solution	Section 8: visual starting point, screen flows for key scenarios S1–S5, mockups of review, unified report and dashboard screens, example API
9	Verification and evaluation plan	How the achievement of the project aim will be demonstrated	Section 9: test levels 9.1, research questions, test bench, matching rule, metrics, baselines, conditions, hypotheses 9.2, threats to validity 9.3
10	Project management and risks	Whether project activities are traceable and the next steps are clearly defined	Section 10: Git repository and YouTrack 10.1, responsibilities 10.2, milestones 10.3, backlog 10.4, risks 10.5, dependencies 10.6; Appendix B: YouTrack backlog
11	Supervisor feedback	Whether the supervisor reviewed the substance of the work and whether that review led to improvements	Section 11: feedback log

B. Minimum requirements for 24 October

Requirement	Where
A completed project README	`README.md` (Appendix A)

Requirement	Where
Project proposal and scope	Section 1
Literature review, existing system comparison, and technology comparison	Sections 3, 4, 5
Initial requirements and acceptance criteria	Section 6
Required architectural diagrams, including editable source files	Section 7; `diagrams/src/*.puml`, `diagrams/export/*.svg/png`
Prototype materials or a link to the prototype	Section 8; `docs/prototype/`
Verification and evaluation plan	Section 9
A link to YouTrack with assigned and prioritized tasks	https://qait-diploma.youtrack.cloud/projects/QAIT (24 prioritized tasks; owner named in each task)
Evidence of progressive contributions from each team member	Commit history and pull requests

C. Git repository requirements

Step	Requirement	How it is met
1	Create the project repository	https://github.com/KNari06/qait-diploma (private)
2	Provide access and identify contributors	Team members invited with write access; supervisor and committee access to be given before the pre-defense
3	Prepare the README	All required sections present
4	Organize the repository	`src/`, `docs/`, `diagrams/`, `tests/`, `config/`, `.gitignore`, `.env.example` with placeholders
5	Record work progressively	One commit per deliverable, descriptive messages
6	Link changes to YouTrack tasks	Task ID in every commit, branch name and pull request (e.g. `QAIT-4-evidence-log`)
7	Use branches and pull requests	Changes go through pull requests with a reviewer from the team
8	Tag each pre-defense submission	Tag `pre-defense-1` set on the submitted version
9	Demonstrate the repository	Trace example: YouTrack QAIT-4 → branch `QAIT-4-evidence-log` → pull request #1 → `docs/04-existing-systems.md`

Pre-defense 1: Submission Report

Project	Development of an AI platform for software quality assurance automation based on large language models
Team	Қайдар Н.Н., Танибергенов Б.Б., Сариев М.Б. (SE-2406)
Supervisor	Айтмұханбетова Э.А.
Repository	https://github.com/KNari06/qait-diploma
Tag	`pre-defense-1` (set after the open pull requests are merged)
YouTrack	https://qait-diploma.youtrack.cloud/projects/QAIT (24 tasks, QAIT-1 … QAIT-24)
Date	24 Oct 2026
PDF	QAIT_PreDefense1_Documentation.pdf: all documents in one file

1. Completed work

#	Component (stage requirement)	Deliverable
1	Project proposal	01-proposal.md: problem, users, aim, 8 objectives, outcomes, scope, complexity
2	Evidence of relevance	02-user-research.md: 7 assumptions, interview guide and template, hypothetical proto-personas, evidence from literature and existing systems
3	Literature review	03-literature-review.md: 20 sources (18 peer-reviewed, most 2020–2025), analysis table, synthesis, gaps
4	Existing system comparison	04-existing-systems.md: 5 systems × 9 criteria, evidence log
5	Technology comparison	05-technology-comparison.md: 6 decisions × 3 alternatives, weighted scoring
6	Requirements	06-requirements.md: 30 functional (4 of them provisional, from proto-personas; PR #3) + 11 non-functional requirements, MoSCoW, acceptance criteria, traceability
7	Architecture and diagrams	07-architecture.md; context, use case, component, ER, 2 sequence diagrams: sources in `diagrams/src` (PlantUML), exports in `diagrams/export`; defect JSON schema in `src/shared`
8	UX / prototype	08-ux-prototype.md: QAIT.dev screen as a visual starting point, screen flow S1–S5, mockups of review (S2), unified report (S4) and dashboard (S5), example API
9	Verification and evaluation plan	09-verification-plan.md: test levels, 4 research questions, test bench (≥100 seeded defects), metrics, baselines, hypotheses, threats
10	Project management and risks	10-project-management.md: roles, 7 milestones, 24-task backlog in YouTrack, 11 risks, dependencies
11	Supervisor feedback	11-supervisor-feedback.md: oral review on 24 Oct 2026, materials in good order

Repository setup: README with all required sections, `docs/`, `diagrams/`, `src/`, `tests/`, `config/.env.example`, `.gitignore`. Commits reference YouTrack task IDs, and changes go through pull requests.

2. Known limitations

- **User research:** the 3–5 interviews required for component 2 have not been conducted yet. Requirements currently rest on the literature, existing systems and the team's assumptions, and doc 02 marks each assumption as supported or unverified.
- **Contributions:** most commits so far are by one member. The other members' pull requests (QAIT-2, QAIT-3) are in progress.

- **Prior work:** the relation between the diploma scope and the existing QAIT.dev product still has to be stated in the README (QAIT-2).
- **YouTrack:** tasks are imported but not yet assigned to members (accounts pending).
- No runnable code is part of this submission (not required at this stage).

3. Outstanding tasks

Task	Owner	Due
Conduct and record 3–5 interviews; update requirements (QAIT-2)	Қайдар Н.Н.	
Verify all literature references (QAIT-3)	Сариев М.Б.	
Describe reuse of QAIT.dev; add pilot observations (QAIT-2)	Таниберген Б.Б.	
Assign YouTrack tasks; give committee access to repositories	Қайдар Н.Н.	before pre-defense
Next stage: skeleton, CI, auth (M2, QAIT-10 … 13)	Таниберген Б.Б., Қайдар Н.Н.	15 Nov 2026

4. Questions for the supervisor

1. Is the scope (code review + browser-agent testing + unified report) right for a 3-person diploma, or should the evaluation focus on one of the two parts?
2. Can prior work on QAIT.dev be reused as a base, and how should this be presented?
3. Which interviewees would be acceptable for component 2 (QAIT.dev pilot clients, industry contacts)?
4. Are the evaluation hypotheses H1–H4 in doc 09 acceptable as the research contribution?

1. Project proposal

Title	Development of an AI platform for software quality assurance automation based on large language models
Programme	Software Engineering
Form	Team diploma project (3 students)
Team	Қайдар Н.Н. (web client, reports) · Таниберген Б.Б. (backend, architecture) · Сариев М.Б. (test bench, evaluation) — group SE-2406
Supervisor	Айтмұханбетова Э.А., senior lecturer, School of Software Engineering

1.1 Problem statement

Quality assurance in small teams relies on two activities: **code review** of changes and **testing** of the running application.

- Code review is manual and slow. Reviewers miss defects when a change is large or when they are not familiar with the code. Studies of modern code review show that most review comments concern maintainability, and only a small share point at functional defects [L3, L4].
- Scripted end-to-end tests break whenever selectors or page structure change, and keeping them working takes a lot of effort [L11, L12]. Small teams therefore often test manually or not at all.
- Static analysers find only part of real defects, and developers ignore them because of false positives [L13, L14].

LLMs can now comment on code [L1, L2, L5] and operate a web browser from a natural-language goal [L9, L10]. Existing products, however, cover **either** review **or** testing, keep the findings in separate tools, and seldom publish how accurate they are (see 04-existing-systems.md).

The problem: there is no single, measurable tool that uses LLMs for both static (code) and dynamic (running application) quality checks and presents the defects found by both in one report.

1.2 Target users

User	Need
Developer in a small team	Get review comments on a pull request in minutes, before a colleague reviews it
QA engineer / manual tester	Describe a scenario in plain language and have it executed, without writing scripts
Team lead	See all defects of a release in one place, ranked by severity

Assumptions about these users are checked through user interviews (02-user-research.md).

1.3 Aim

To design, implement and evaluate a web platform that automates code review and web-application testing with large language models and joins their findings into a single defect report.

1.4 Objectives

#	Objective	Owner	Result
O1	Study users' review and testing practices	All	Interview findings, requirements

#	Objective	Owner	Result
O2	Review literature and existing systems	All	Docs 03, 04
O3	Design architecture and common defect format	Бекнұр	Doc 07, schema
O4	Implement code-review service with GitHub integration	Бекнұр	Service + API
O5	Implement browser-agent testing service	Бекнұр, Максот	Service + API
O6	Implement web client, unified report, dashboard	Нариман	Web app
O7	Build test bench with seeded defects	Максот	Bench repositories and apps
O8	Evaluate accuracy vs. baseline	Максот	Precision / recall / F1 results

1.5 Expected outcomes

1. A deployed web platform where a user connects a GitHub repository, starts a review of a pull request, runs natural-language test scenarios against a URL and reads one defect report.
2. A common defect format used by both services (type, severity, location in code or step, evidence: screenshot, log, diff hunk).
3. A test bench of at least 10 repositories / web apps with at least 100 seeded defects of known type.
4. Experimental results: precision, recall and F1 of code review, of testing, and of both combined, compared with a baseline (static analysers for code; scripted Playwright tests for UI).

1.6 Scope

In scope

- GitHub repositories, connected through a GitHub App (webhooks on pull requests).
- Review of pull-request diffs with repository context (changed files, related files).
- Web applications reachable by URL; scenarios written in English or Russian.
- OpenAI and Gemini models behind one provider interface.
- Unified report, run history, dashboard with experiment statistics.
- Evaluation bench and method.

Out of scope

- GitLab, Bitbucket, Azure DevOps.
- Native mobile applications.
- Performance, load and penetration testing.
- Training or fine-tuning own models.
- Billing and multi-organisation administration.

1.7 Engineering complexity

- Several independent services communicating through REST and an asynchronous job queue.
- Integration with an external platform (GitHub App, webhooks, checks / comments API).
- LLM engineering: prompt design, retrieval of repository context within token limits, structured (JSON-schema) output, cost control.
- Control of a real browser by an LLM agent: page-state extraction, action selection, recovery from failed actions.
- Experimental evaluation on a purpose-built bench with statistical comparison.

1.8 Milestones

Date	Milestone
24 Oct 2026	Pre-defense 1: analysis, design, plan (this submission)
	Pre-defense 2: working code-review and testing services, report, first experiments
	Final pre-defense: full evaluation, explanatory note

Detailed plan: 10-project-management.md.

2. Evidence of the relevance

Status: **guide ready; proto-personas (2.5) and evidence from literature and prior work (2.6) ready; interviews to be conducted (2.7).**

2.1 Goal

Check the assumptions behind the requirements:

ID	Assumption	Requirements that depend on it
A1	Code review delays merging in small teams	FR-CR1, NFR-P1
A2	Teams have few or no automated UI tests because scripts are costly to maintain	FR-T1, FR-T2, FR-T8
A3	Review and test results live in different tools and are hard to see together	FR-R2
A4	Users will not trust AI findings without evidence and a known accuracy	FR-CR4, FR-T3, FR-T7, FR-R3, FR-R5
A5	Teams are willing to send code to a cloud LLM (or need a self-hosted option)	FR-CR7, FR-P5, NFR-S1
A6	When a test fails, users need a plain-language reason with a screenshot, not a stack trace	FR-T3
A7	Cost per run must be visible and limited	NFR-C1, FR-P6, FR-T5

2.2 Participants

3–5 people. Participants are anonymised and referred to as P1…P5 with role and context only.

Target profile: a developer, QA engineer or team lead in a team of ≤ 20 people who personally performs code review or testing. Classmates without work experience and supervisors or teachers do not qualify solely through their academic role.

Possible sources: QAIT.dev pilot clients, companies where team members have worked or interned, developers and QA engineers in Astana IT companies.

2.3 Protocol

- 15–20 minutes, online, by voice call or in person, in Kazakh, Russian or English.
- Ask for oral consent to take notes. No recording without separate consent.
- Write notes on the same day; record the actual interview date, duration, interviewer and consent.
- Ask about current work and specific recent examples before introducing QAIT. Use follow-up prompts selectively to fit the time available.
- Record negative answers as well as positive ones. Missing information is **unclear**, not a rejection or confirmation. Separate what the participant said or showed from the interviewer's interpretation.

Core questions

1. **Context:** What is your role, how large is your team, and what do you build? Which review or testing tasks do you personally perform?
2. **Code review (A1):** How does code review work? How many PRs are waiting, and how long do they wait? Walk me through a recent PR: did review delay merging or release?

3. **Review outcomes (A4):** What problems did review catch recently, and what slipped through? Can you give a concrete example of each?
4. **UI testing (A2):** How did you test the interface before your last release? Which scenarios were manual or automated, how long did it take, and how many people were involved?
5. **Automation and diagnosis (A2, A6):** If you have UI tests, how many failed without a product bug during the last release or another stated period, out of how many runs? Who fixes them? Walk me through the last failed run: what evidence did you use and how long did diagnosis take? If you have no UI tests, what prevents you from adding them?
6. **Reporting (A3):** Where are defects, review findings and test results recorded? How many tools do you use? Describe a recent case where information was hard to find or transfer, if any.
7. **Trust (A4, A6):** What evidence would let you check an AI-reported defect? When would you reject its conclusion? If you have used AI tools, describe a recent finding you accepted or rejected and why.
8. **Data policy (A5):** Are you allowed to send code, DOM, screenshots or logs to an external AI provider? What must be excluded, who approves this, and where are the rules recorded? If the policy is unknown, record that rather than assuming permission.

Follow-up and concept discussion

Only after the core questions, briefly explain QAIT as a proposed tool for code review and browser-based testing with linked evidence. Do not promise accuracy or time savings.

- **Scenario authoring (PP3):** How would you prefer to create a scenario? What would you need to inspect or edit before running AI-generated steps? Which language would you use?
- **Element choices (PP1, A4):** If an agent selects a different element after a page change, what would you need to see to judge that choice? What should happen if the intended element is missing?
- **Cost (A7):** How do you currently track and limit spending on testing or AI tools? Have unexpected costs occurred? What per-run limit or approval would you need? Record any proposed price as a stated preference, not a purchase commitment.
- **Workflow fit (A3):** Which existing workflow or integration would this need to fit? What would prevent adoption? A positive reaction to the concept does not confirm a current workflow problem.

2.4 Interview notes template

```
Participant: P_  | Role: _______ | Team size: __ | Domain: _______ | Date: ____ | Interviewer: ____
Format / actual duration: _______ | Consent to notes: _______ | Notes written on: _______
Key quotes (verbatim, short):
Observations (what the participant said or showed; include recent examples):
Interviewer interpretation / open questions:
Assumptions:  A1 confirmed / rejected / unclear – because …
              A2 …
              A3 …
              A4 …
              A5 …
              A6 …
              A7 …
New requirements or changes:
```

2.5 Proto-personas (hypothetical)

> These personas are **hypothetical**. The team wrote them from its own experience in QA and from the literature. They are **not interview data**. They describe whom the interviews should reach and what we expect to hear. Each one is checked against the real interviews in 2.7.

PP1: QA lead in a product company

- **Context:** leads 5–7 testers, mostly manual regression of 300+ cases before each release; two automation engineers.
- **Expected pains:** regression takes days; scripted tests break when UI classes or structure change, so automation engineers spend much of their time fixing tests; triaging a failed run (bug, flaky test or environment) is slow.
- **Expected needs:** testers who know the product write automated scenarios in plain language; every automatic "fix" or element choice is shown with its reason and confidence; failed runs come with the failing request, log and screenshot.
- **Expected concerns:** a tool that turns a real bug green by acting on the wrong element; safety of test credentials and rules for sending code, DOM, screenshots and logs to external models; LLM cost per run and a spending cap.
- **Assumptions:** A2, A4, A5, A6, A7 · **Requirements:** FR-T1–T3, FR-T5, FR-T7, FR-P5, FR-P6, NFR-S1, NFR-C1

PP2: CTO of a small team without QA

- **Context:** 10–15 developers, no QA role; developers write unit tests, the product owner checks end-to-end flows by hand before release; some bugs reach users.
- **Expected pains:** hiring a test-automation engineer is expensive; nobody owns UI testing.
- **Expected needs:** checks run automatically on every pull request; one place to see what broke; notifications where the team already talks.
- **Expected concerns:** unpredictable cost and the need for a per-run spending cap; time to set up.
- **Assumptions:** A1, A3, A7 · **Requirements:** FR-CR1, FR-R2, FR-P6, NFR-U1, NFR-C1

PP3: Manual tester moving towards automation

- **Context:** 1–3 years of manual testing; started learning Selenium / JavaScript but has no time between projects.
- **Expected pains:** high entry barrier to automation.
- **Expected needs:** write or edit test steps in their own language, see each step, and fix steps the tool misunderstood; failures explained in words with a screenshot, not a stack trace.
- **Expected concerns:** being replaced; expects the tool to take over routine work, not the judgement of what to test.
- **Assumptions:** A2, A6 · **Requirements:** FR-T1, FR-T3, FR-T4, FR-T8

2.6 Evidence available before the interviews

Status of each assumption from published studies, existing products and the team's own work on QAIT.dev. None of this is interview data. It shows which assumptions have outside support and which still need users to confirm them.

Assumption	Published evidence	Existing systems	Team observation (QAIT.dev pilots)	Status
A1 Review delays merging	Understanding the change is the main review difficulty [L3]; review speed is a core expectation [L4]	LLM reviewers sell on review time (doc 04)		Supported by literature; not yet checked with users
A2 Few UI tests, scripts costly	Locators are the main cause of record/replay test breakage [L11]; robust locators help only partly [L12]	testRigor and QAIT.dev are built on this premise		Supported by literature

Assumption	Published evidence	Existing systems	Team observation (QAIT.dev pilots)	Status
A3 Results scattered across tools	No direct study found	No tool in doc 04 combines code and UI findings		**Unverified**: main question for the interviews
A4 Trust needs evidence and accuracy	False positives and poor explanations drive developers away from static tools [L13]; acceptance rises when output is verified [L20]	Review tools publish no accuracy figures (doc 04, C7)		Supported by literature
A5 Cloud LLM acceptable	No direct study	Most products are SaaS-only (C9)		**Unverified**
A6 Plain-language failure reasons	Explanations matter for tool adoption [L13]	QAIT.dev, testRigor give screenshots and step text		Partly supported
A7 Visible cost per run	No direct study	—		**Unverified**

2.7 Interview findings

Assumption	P1	P2	P3	P4	P5	Verdict
A1						
A2						
A3						
A4						
A5						
A6						
A7						

Changes to requirements

3. Literature review

3.1 Method

- **Questions.** (RQ-L1) How well can models generate useful code-review comments? (RQ-L2) How well can LLM agents test applications through their user interface? (RQ-L3) How is defect-detection accuracy measured, and what baselines are used?
- **Sources.** ACM Digital Library, IEEE Xplore, arXiv, Google Scholar.
- **Search terms.** "automated code review", "review comment generation", "LLM software testing", "GUI testing LLM", "web agent benchmark", "test flakiness / breakage", "static analysis false positives".
- **Inclusion.** Peer-reviewed papers or widely cited preprints. Most are from 2020–2025, because the field changes fast. Foundational empirical studies on code review, web-test breakage and static analysis are kept even when older.
- **Result.** 20 sources are used. 18 of them are peer-reviewed scientific publications and 2 are industrial preprints.

> Team note: check the bibliographic details of every entry (authors, pages, DOI) against the publisher page before submission.

3.2 Analysis table

ID	Reference	Year	Type	Problem / approach	Key finding	Limitation	Use in this project
L1	Tufano R. et al., "Using Pre-Trained Models to Boost Code Review Automation", ICSE	2022	Conf.	Pre-trained T5 for review-comment and code-refinement tasks	Pre-training clearly improves review automation over non-pre-trained models	Java only; exact-match metrics	Baseline idea for comment generation; metric discussion
L2	Li Z. et al., "Automating Code Review Activities by Large-Scale Pre-training" (CodeReviewer), ESEC/FSE	2022	Conf.	Model pre-trained on diff-review pairs, 9 languages	Defines quality estimation, comment generation and refinement tasks with a public dataset	BLEU-based evaluation correlates poorly with usefulness	Public dataset for the evaluation bench; task decomposition
L3	Bacchelli A., Bird C., "Expectations, Outcomes, and Challenges of Modern Code Review", ICSE	2013	Conf.	Interviews and analysis at Microsoft	Finding defects is the main expectation, yet most comments are about maintainability; understanding the change is the main difficulty	Single company	Motivation; requirement to give context with every comment
L4	Sadowski C. et al., "Modern Code Review: A Case Study at Google", ICSE-SEIP	2018	Conf.	Large-scale study of review at Google	Review is fast and frequent; main value is understanding and maintainability	Large-company context	Workflow expectations (review within hours, small changes)

ID	Reference	Year	Type	Problem / approach	Key finding	Limitation	Use in this project
L5	Lu J. et al., "LLaMA-Reviewer: Advancing Code Review Automation with LLMs through Parameter-Efficient Fine-Tuning", ISSRE	2023	Conf.	LoRA / prefix tuning of LLaMA for review	A 6.7B model with <1% parameters tuned matches specialised models	Fine-tuning needed; offline metrics	Justifies prompting large hosted models instead of fine-tuning (out of scope)
L6	Tufano R. et al., "Code Review Automation: Strengths and Weaknesses of the State of the Art", arXiv:2401.05136	2024	Preprint (peer-reviewed version in TSE)	Manual inspection of 2,296 predictions of 3 tools	Taxonomy of changes where tools succeed or fail; tools rarely find functional defects	Java only	Defect taxonomy for the bench; motivates measuring defect detection, not text similarity
L7	"BitsAI-CR: Automated Code Review via LLM in Practice", arXiv:2501.15134 (ByteDance)	2025	Industrial preprint	Two-stage pipeline: rule-based detection + LLM filter	75% precision of comments in production; the "outdated rate" shows whether developers act on comments	Company-specific rules	Two-stage design (generate, then filter) to raise precision; adoption metric
L8	Wang J. et al., "Software Testing with Large Language Models: Survey, Landscape, and Vision", IEEE TSE	2024	Journal	Survey of 100+ papers on LLMs in testing	LLMs are used for unit-test generation, test oracles, input generation and GUI testing; evaluation practices vary widely	Survey; field moves fast	Classification of the testing service; evaluation design
L9	Zhou S. et al., "WebArena: A Realistic Web Environment for Building Autonomous Agents", ICLR	2024	Conf.	Self-hosted realistic web apps for agent tasks	Best GPT-4 agent completes ~14% of tasks vs ~78% for humans	Task completion, not defect finding	Shows agents are unreliable; need retries, step limits and human-readable traces
L10	Deng X. et al., "Mind2Web: Towards a Generalist Agent for the Web", NeurIPS	2023	Conf.	Dataset of 2,000+ tasks on 137 websites	Filtering the DOM to candidate elements before LLM selection improves accuracy and cost	Offline evaluation	Page-state reduction (candidate elements) for the browser agent
L11	Hammoudi M., Rothermel G., Tonella P., "Why Do Record/Replay Tests of Web Applications Break?", ICST	2016	Conf.	1,065 breakages in 453 versions of web apps	Most breakages come from locators	Selenium IDE tests	Motivation: natural-language steps resolved at run time instead of fixed locators
L12	Leotta M. et al., "ROBULA+: An Algorithm for Generating Robust XPath Locators for Web Testing", J. Software: Evolution and Process	2016	Journal	Algorithm for robust locators	Robust locators reduce, but do not remove, breakages	Still locator-based	Baseline comparison point for UI robustness

ID	Reference	Year	Type	Problem / approach	Key finding	Limitation	Use in this project
L13	Johnson B. et al., "Why Don't Software Developers Use Static Analysis Tools to Find Bugs?", ICSE	2013	Conf.	Interviews with developers	False positives and poor explanations drive developers away	Small sample	Requirement: every finding must explain why and where; severity ranking
L14	Habib A., Pradel M., "How Many of All Bugs Do We Find? A Study of Static Bug Detectors", ASE	2018	Conf.	3 static detectors on 594 real Java bugs (Defects4J)	Detectors find only ~4.5% of the bugs	Java; older tools	Baseline: static analysers for the code-review experiments
L15	Liu Z. et al., "Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions", ICSE	2024	Conf.	LLM chooses GUI actions with memory of tested functions	+32% activity coverage and 31% more bugs than the best baseline on 93 apps	Mobile, not web	Memory / coverage prompting for the browser agent
L16	Hou X. et al., "Large Language Models for Software Engineering: A Systematic Literature Review", ACM TOSEM	2024	Journal	SLR of 395 papers	Maps LLM uses across SE tasks; notes data leakage and evaluation threats	Rapidly ageing	Threats to validity (data contamination) in the evaluation plan
L17	Jimenez C. E. et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?", ICLR	2024	Conf.	Benchmark of 2,294 real GitHub issues	Repository-level context is the main difficulty for models	Issue resolution, not review	Context retrieval from the repository for review
L18	Yao S. et al., "ReAct: Synergizing Reasoning and Acting in Language Models", ICLR	2023	Conf.	Interleaving reasoning and tool actions	Better reliability and interpretability on interactive tasks	General tasks	Agent loop design (observe, reason, act)
L19	Lewis P. et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", NeurIPS	2020	Conf.	Retrieval combined with generation	Retrieval improves factuality	Not code-specific	Retrieval of related files for review prompts
L20	Alshahwan N. et al., "Automated Unit Test Improvement using Large Language Models at Meta", FSE Companion	2024	Conf. (industry)	LLM-generated tests filtered by build/run checks	75% of generated cases built correctly; 73% of recommendations accepted by engineers	Unit tests only	"Generate then verify" principle: findings from the agent are kept only with evidence

3.3 Synthesis

Code review (L1–L7, L17, L19). Review automation moved from task-specific pre-trained models (L1, L2) to tuned or prompted LLMs (L5, L7). Two problems keep appearing. First, evaluation by text similarity (BLEU, exact match) says little about whether a comment points at a real defect (L2, L6). Second, models lack repository context (L17). Industrial systems reach useful precision only when a second filtering stage removes noisy comments (L7). Studies of human review (L3, L4) show that developers value comments that explain the change and its risk. *Consequence for the project:* the review service retrieves related files (L19), returns structured

findings with location and rationale, filters them in a second pass, and is evaluated on seeded defects with precision / recall rather than BLEU.

UI testing with agents (L8–L12, L15, L18). Scripted web tests break mostly because of locators (L11), and robust locators help only partly (L12). LLM agents can choose actions from a description (L15, L18) and simplified page state improves their accuracy (L10). Their success rate on realistic tasks is still low (L9). *Consequence:* test steps are written in natural language and resolved at run time. The agent works step by step with a step limit and retries, and keeps screenshots and logs as evidence. The evaluation reports both detection accuracy and run stability.

Static analysis and trust (L13, L14, L20). Static tools find few real bugs (L14), and developers stop using tools with many false positives (L13). Industrial LLM tools that verify their output gain acceptance (L20). *Consequence:* the baseline for code is a static analyser. Each finding carries severity and evidence, and precision is a primary metric.

3.4 Gaps addressed by this project

1. Review and UI-testing research is evaluated separately. There is little evidence on how much the two approaches **overlap or complement** each other on the same system.
2. Evaluation of review tools rarely uses **seeded defects with known ground truth**.
3. Practical tools seldom publish **accuracy figures** that a small team can use to decide whether to trust them.

3.5 Additional technical sources (not counted as scientific)

- Playwright documentation, https://playwright.dev/docs/intro
- GitHub Docs, "About creating GitHub Apps" and "Webhook events and payloads", https://docs.github.com/en/apps
- OpenAI API, "Structured Outputs", https://platform.openai.com/docs/guides/structured-outputs

4. Existing system comparison

4.1 Selected systems

Five systems were chosen. Each covers at least one of the platform's two functions:

System	Category	Why selected
CodeRabbit	LLM pull-request review	Widely used LLM reviewer for GitHub / GitLab
GitHub Copilot code review	LLM pull-request review	Built into GitHub, the platform our users already use
SonarQube / SonarCloud	Static analysis	Common non-LLM quality gate; represents the baseline approach
testRigor	AI / natural-language E2E testing	Tests written in plain English, no locators
QAIT.dev	AI web and mobile testing	Team members' prior work; LLM-based test recording and execution

Mabl and Applitools were also considered, but they were not added as separate rows. Mabl is close to testRigor (low-code E2E with self-healing). Applitools is focused on visual comparison.

4.2 Criteria

#	Criterion	Why it matters (source)
C1	Reviews code changes (pull requests)	Static quality check
C2	Tests the running web application	Dynamic quality check
C3	Tests defined in natural language, no locators	Locator breakage is the main cause of test failures [L11]
C4	Uses repository context beyond the diff	Context is the main difficulty for models [L17]
C5	Single report joining code and UI findings	Interviews (to be confirmed, see doc 02)
C6	Evidence per finding (location, screenshot, log)	Trust and false positives [L13]
C7	Published / measurable accuracy	Users cannot judge trust otherwise [L6, L7]
C8	Choice of LLM provider	Cost and data policy of the team
C9	Self-hosting possible	Data stays in the company's infrastructure

4.3 Comparison

Legend: ✓ supported · ◐ partial · ✗ not supported · ? not documented

Criterion	CodeRabbit	Copilot review	SonarQube	testRigor	QAIT.dev	**Proposed platform**
C1 PR review	✓	✓	◐ rule-based PR analysis + LLM fix suggestions (AI CodeFix)	✗	✗	✓
C2 Web app testing	✗	✗	✗	✓	✓	✓

Criterion	CodeRabbit	Copilot review	SonarQube	testRigor	QAIT.dev	**Proposed platform**
C3 Natural-language tests	✗	✗	✗	✓	✓	✓
C4 Repository context	✓	◐ (instruction files: `copilot-instructions.md`, `AGENTS.md`)	✓ (rules over whole project)	n/a	n/a	✓
C5 Unified code + UI report	✗	✗	✗	✗	✗	✓
C6 Evidence per finding	✓ (code)	✓ (code)	✓ (code)	✓ (screens)	✓ (screens, logs)	✓ (both)
C7 Published accuracy	✗ (none found in docs)	✗ (none found in docs)	◐ (rule docs)	✗ (vendor marketing claims only)	◐ (team's paper vs. Playwright)	✓ (this project's evaluation)
C8 LLM choice	◐ (only self-hosted: bring your own model)	✗ (model switching not supported for code review)	n/a	?	◐ (OpenAI / Gemini)	✓
C9 Self-hosting	◐ (Enterprise, 500+ seats)	✗	✓ (SonarQube Server)	◐ (on-premise mentioned by third-party listings, not in public docs)	✗	✓ (Docker)

Cells were checked against public vendor documentation on 24 Oct 2026 (see 4.5). Absence (✗) means the feature or figure was not found in public documentation. QAIT.dev cells are based on the team's own work on the product.

4.4 Conclusions

- No system in the sample covers **both** code review and UI testing. A team has to buy and configure two tools and read two reports (C1 + C2 + C5).
- LLM review tools do not publish accuracy on a controlled defect set (C7). Static analysers have well-defined rules but low recall on real bugs [L14].
- Natural-language E2E tools solve locator brittleness (C3), but they are mostly SaaS without a documented choice of model (C8, C9).
- LLM review tools hide or restrict model choice: Copilot code review does not support model switching, and CodeRabbit allows its own model only in self-hosted Enterprise (C8).
- **Need for the proposed solution:** one tool that runs both checks, produces one ranked defect report with evidence, and states its own measured accuracy.

4.5 Evidence log

System	Cells	Source	Checked on	By
CodeRabbit	C1, C4, C8, C9	Self-hosted overview: https://docs.coderabbit.ai/self-hosted/overview	24 Oct 2026	Қайдар Н.Н.

System	Cells	Source	Checked on	By
GitHub Copilot code review	C1, C4, C8	Concepts — code review: https://docs.github.com/en/copilot/concepts/agents/code-review ; tutorial "Use Copilot code review across the pull request lifecycle": https://docs.github.com/en/copilot/tutorials/use-copilot-code-review-across-the-pull-request-lifecycle	24 Oct 2026	Қайдар Н.Н.
SonarQube	C1, C4, C6, C9	Pull request analysis: https://docs.sonarsource.com/sonarqube-cloud/analyzing-source-code/pull-request-analysis ; AI CodeFix: https://www.sonarsource.com/blog/ai-codefix-sonar	24 Oct 2026	Қайдар Н.Н.
testRigor	C2, C3, C6, C9	Features: https://testrigor.com ; deployment options per third-party listing (Gartner Peer Insights): https://www.gartner.com/reviews/product/testrigor	24 Oct 2026	Қайдар Н.Н.
QAIT.dev	all	Team's own work on the product; https://qait.dev	24 Oct 2026	Танибергенов Б.Б. (to confirm)

Open points: testRigor model choice (C8) and on-premise details (C9) are not described in public docs and should be asked from the vendor.

5. Technology comparison

Each decision is scored 1–5 against requirements from 06-requirements.md. Weights reflect project priorities.

T1. Web client

Criterion (weight)	React + TS	Vue 3 + TS	Angular
Team experience (3)	5	2	2
Ecosystem for charts / tables (2)	5	4	4
Real-time updates via WebSocket (1)	5	5	5
Learning / boilerplate cost (1)	4	5	2
Weighted total	**34**	24	21

Decision: React with TypeScript and Vite. The frontend developer works with React commercially, and charting libraries (Recharts, ECharts) and table components are mature. Supports NFR-U1, FR-R1..R5.

T2. Backend platform

Criterion (weight)	Node.js (TS, Express)	Python (FastAPI)	Java (Spring Boot)
Same language as client, shared types (3)	5	2	2
LLM SDKs (OpenAI, Gemini) (2)	5	5	3
Playwright support (2)	5 (native)	4	4
Async I/O for many waiting LLM calls (2)	5	5	4
Team experience (2)	5	3	2
Weighted total	**55**	40	32

Decision: Node.js with TypeScript for all services. A shared package holds the defect schema used by every service (FR-D1). Python stays an option for the evaluation scripts (pandas), which are not part of the runtime.

T3. Data storage

Criterion (weight)	PostgreSQL	MongoDB	SQLite
Relational model: project → run → finding (3)	5	3	5
JSONB for model output and evidence (2)	5	5	3
Concurrent writes from several services (2)	5	5	1
Aggregation queries for dashboard (2)	5	4	4
Managed hosting options (1)	5	5	1
Weighted total	**50**	42	32

Decision: PostgreSQL. Findings are strongly relational, and reports and dashboards need joins and grouping (FR-R3, FR-R5). JSONB stores raw model output. Screenshots and logs go to object storage (S3-compatible), and only their URLs are kept in the database.

T4. Service interaction

Criterion (weight)	REST only (synchronous)	REST + Redis job queue (BullMQ)	RabbitMQ / Kafka
Long jobs (review 1–5 min, test runs up to 15 min) (3)	1	5	5
Retries and back-off for LLM / network errors (2)	2	5	4
Operational simplicity for a 3-person team (2)	5	4	2
Progress events to UI (1)	2	4 (with WebSocket)	4
Weighted total	19	**37**	31

Decision: REST for client ↔ gateway and synchronous queries. Review and test jobs go through a Redis-backed queue with retries (NFR-R1). Progress reaches the client by WebSocket (FR-T4). Kafka's throughput is not needed at this scale.

T5. Browser automation

Criterion (weight)	Playwright	Selenium WebDriver	Puppeteer
Accessibility tree / DOM snapshot for the agent (3)	5	3	4
Cross-browser (Chromium, Firefox, WebKit) (1)	5	5	2
Auto-waiting, fewer flaky steps (2)	5	2	3
Trace, screenshot, video built in (2)	5	2	3
Headless in Docker (1)	5	4	5
Weighted total	**45**	26	31

Decision: Playwright. Its accessibility snapshot gives the LLM a compact page description [L10], and its trace and screenshots serve as evidence (FR-D2).

T6. LLM provider

Criterion (weight)	OpenAI (GPT family)	Google Gemini	Self-hosted open model (e.g. Llama via vLLM)
Code review quality (3)	5	4	3
Structured JSON output (2)	5	4	3
Long context for repository files (2)	4	5	3
Cost per run (2)	3	4	4 (needs GPU)
Data stays on premises (1)	1	1	5
Weighted total	40	39	34

Decision: no single provider. A provider interface (`LLMProvider`) with OpenAI and Gemini adapters lets the experiments compare models (Objective O8, NFR-M2). A self-hosted model is a possible extension. It is not required.

Summary

Decision	Choice
T1 Web client	React + TypeScript
T2 Backend	Node.js + TypeScript (Express)

Decision	Choice
T3 Storage	PostgreSQL + S3-compatible object storage
T4 Interaction	REST + Redis/BullMQ queue + WebSocket
T5 Browser	Playwright
T6 LLM	Pluggable: OpenAI, Gemini

6. Requirements specification

Priorities use MoSCoW: **M** must, **S** should, **C** could. Every **M** requirement has an acceptance criterion that can be checked by a test, a demonstration or a measurement. The "Source" column links each requirement to evidence: **I** interviews (doc 02, to be confirmed), **L** literature (doc 03), **E** existing systems (doc 04), **A** team assumption, **PP** proto-persona (hypothetical, doc 02 §2.5).

PP1–PP3 are hypothetical sources, not verified interview evidence. Requirements derived from them and their priorities remain provisional until checked with real participants in doc 02 §2.7.

6.1 Actors

Actor	Description
User	Developer, QA engineer or team lead with an account
GitHub	External platform: sends webhooks, receives review comments
LLM provider	External API (OpenAI, Gemini)
Target web app	Application under test, reachable by URL

6.2 Functional requirements

Accounts and projects (P)

ID	Requirement	Pri	Acceptance criterion	Source
FR-P1	User signs in with GitHub OAuth	M	A new user signs in with GitHub. A user record is created and the user lands on the project list	A
FR-P2	User creates a project and links a GitHub repository through the GitHub App	M	After installing the App, the repository appears in the project. The project shows "connected"	E
FR-P3	User adds one or more target URLs (environments) to a project	M	URL is validated (http/https, reachable). It appears in the environment list	A
FR-P4	User stores test credentials for a target app as an encrypted secret	S	Secret is never returned by the API in plain text. It is used by the agent at run time	A
FR-P5	User configures which data may be sent to an external LLM per project; unapproved categories and secrets are blocked	M	With no approved policy, no external LLM request is made. For a policy allowing code but denying DOM, screenshots and logs, captured outbound requests contain only allowed categories and exclude seeded API keys, tokens and test credentials; a run needing blocked data stops with an explanation	PP1
FR-P6	User sets an LLM spending cap per run, enforced for both code-review and testing runs	M	Using fixed provider prices, a run whose next call would exceed the cap stops as "aborted (budget)" before that call. The check includes retries and reserved maximum costs of in-flight calls; the run shows the cap and recorded cost (NFR-C1). If a call's maximum cost cannot be bounded, it is not sent	PP1, PP2

Code review (CR)

ID	Requirement	Pri	Acceptance criterion	Source
FR-CR1	Review starts automatically when a pull request is opened or updated	M	Opening a PR in a connected repo creates a review job within 10 s of the webhook	E, I

ID	Requirement	Pri	Acceptance criterion	Source
FR-CR2	User can start a review manually for a PR or a branch	M	"Run review" in the UI creates a job. Its status is visible	A
FR-CR3	The service sends the diff plus related repository files within a token budget	M	Prompt log for a test PR contains changed files and at least one imported/related file, and stays within the configured limit	L17, L19
FR-CR4	Each finding has type, severity, file, line range, explanation and suggested fix	M	Output validates against the defect JSON schema. Invalid output is retried or rejected	L13
FR-CR5	A second filtering pass removes low-confidence or duplicate findings	S	With filter on, the number of findings on the bench decreases and precision does not drop	L7
FR-CR6	Findings are posted as PR review comments on GitHub	S	Comments appear on the correct lines of the PR	E
FR-CR7	User chooses LLM provider and model per project	M	Changing the model in settings changes the model recorded on the next run	L, A

Testing (T)

ID	Requirement	Pri	Acceptance criterion	Source
FR-T1	User writes a test scenario as natural-language steps and an expected result	M	A scenario with ≥1 step and an expected result is saved and listed	L11, E
FR-T2	The agent executes each step in a real browser and decides actions from page state	M	Login scenario on the bench app passes without any locator written by the user	L10, L18
FR-T3	A failed step or unmet expectation produces a finding with screenshot, console log and step trace	M	Seeded UI defect produces a finding whose evidence includes screenshot and failing step	L20
FR-T4	Run progress is shown live (current step, screenshot)	S	UI updates step status without page reload during a run	A
FR-T5	Runs are limited by step count and time	M	A scenario that cannot finish stops at the limit with status "aborted (limit)"	L9
FR-T6	Agent can generate test scenarios by exploring a page	C	From a URL, the agent proposes ≥3 scenarios that the user can save	L15
FR-T7	Every agent element choice or automatic recovery is shown with its reason and confidence; recovery must not hide a missing intended element or an unmet expectation	M	The trace shows each selected element, reason and confidence, plus original and replacement choices for recovery. In a seeded test with the intended element removed and a similar distractor present, the agent does not act on the distractor or report a pass; the original failure and evidence remain visible. Low-confidence choices stop for user review	PP1
FR-T8	User can inspect and edit AI-generated steps and expected results before approving execution	M	A generated scenario remains a draft until approved. The user changes a step and expected result, saves and approves it, and the run executes that approved version. Regenerating or editing a draft requires approval of the new version before execution	PP3

FR-T7 addresses risk R9 (false passes from choosing the wrong element) in doc 10. FR-T8 applies whenever AI-generated scenarios are available; it does not raise optional exploration (FR-T6) from C to M.

Defects and reports (D, R)

ID	Requirement	Pri	Acceptance criterion	Source
FR-D1	Code-review and testing findings use one defect schema	M	Both services' outputs validate against `defect.schema.json`	A

ID	Requirement	Pri	Acceptance criterion	Source
FR-D2	Evidence (screenshots, logs, diff hunks) is stored and linked to the finding	M	Opening a finding shows its evidence. The file loads from storage	L13
FR-R1	Run history per project with status, duration, model, number of findings	M	All runs of a project are listed and can be filtered by type and status	A
FR-R2	Unified report for a release / commit range shows code and UI findings together	M	Report for a bench release contains findings from both services, sorted by severity	E, I
FR-R3	User marks a finding as confirmed / false positive / fixed	M	Status change is saved and counted in statistics	L7
FR-R4	Report export to PDF or Markdown	S	Exported file contains all findings of the report	A
FR-R5	Dashboard: findings by type and severity, precision from user labels, trend over runs	S	Charts render from stored data for a project with ≥5 runs	A

Evaluation support (EV)

ID	Requirement	Pri	Acceptance criterion	Source
FR-EV1	Runs can be executed in batch over the bench with a fixed configuration	M	A CLI command runs all bench items and stores results with a configuration ID	L6
FR-EV2	Results are exported to CSV for analysis	M	CSV contains one row per finding with run, configuration and ground-truth match	A

6.3 Non-functional requirements

ID	Category	Requirement	Pri	Acceptance criterion
NFR-P1	Performance	Review of a PR ≤ 500 changed lines finishes in ≤ 3 min (p90)	M	Measured over bench PRs
NFR-P2	Performance	API responses (non-job) ≤ 500 ms (p95)	S	Load test with 20 concurrent users
NFR-R1	Reliability	Failed LLM / network calls retried up to 3 times with back-off; job never lost on service restart	M	Kill the worker during a job, then the job resumes or fails visibly
NFR-R2	Reliability	Same scenario on an unchanged app gives the same verdict in ≥ 90% of 10 repeated runs	M	Stability test in the verification plan
NFR-S1	Security	Secrets (API keys, test credentials, GitHub tokens) encrypted at rest; never logged	M	Code review + log inspection; secret scan in CI
NFR-S2	Security	GitHub webhooks verified by signature	M	Request with wrong signature is rejected (401)
NFR-S3	Security	Users see only their own projects	M	API test: access to another user's project returns 403
NFR-U1	Usability	A new user connects a repo and runs the first review in ≤ 10 min without help	S	Usability test with 5 participants
NFR-M1	Maintainability	Services containerised. `docker compose up` starts the full system	M	Fresh machine start following README
NFR-M2	Maintainability	LLM provider is behind an interface. Adding a provider needs no change in services	M	Second adapter added without touching review / testing logic
NFR-C1	Cost	Token usage and cost recorded per run	S	Each run shows tokens and estimated cost

6.4 Traceability (summary)

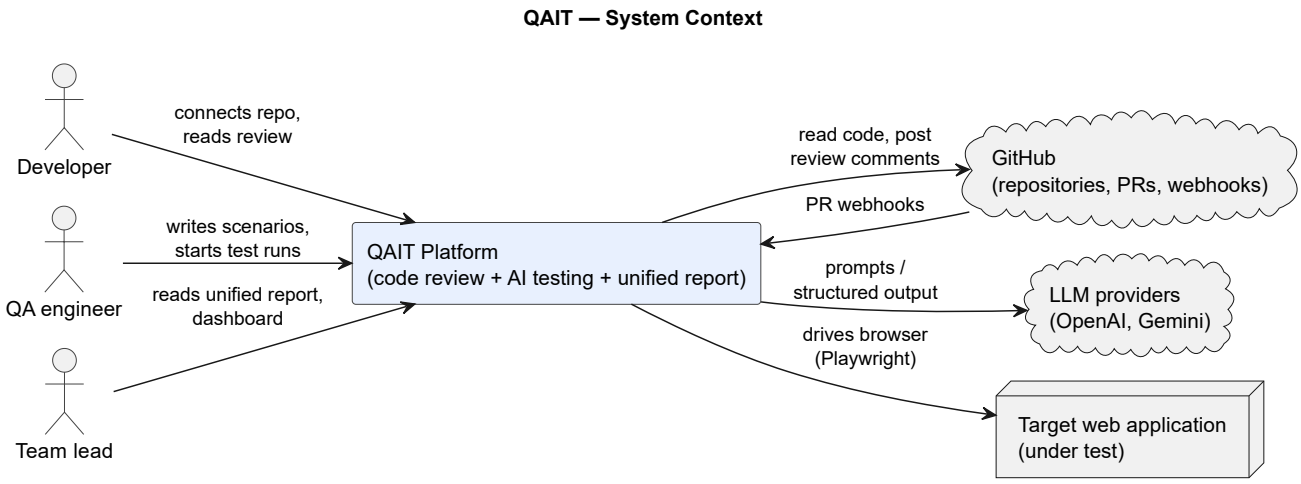
Objective	Requirements
O4 Code review	FR-CR1..7, FR-P5..6, FR-D1..2, NFR-P1
O5 Testing	FR-T1..8, FR-P5..6, FR-D1..2, NFR-R2
O6 Client, report	FR-P1..6, FR-R1..5, NFR-U1
O8 Evaluation	FR-EV1..2, FR-R3, NFR-C1

7. Architecture and diagrams

Editable sources are in `diagrams/src` (PlantUML) and the exports in `diagrams/export`. To re-render:

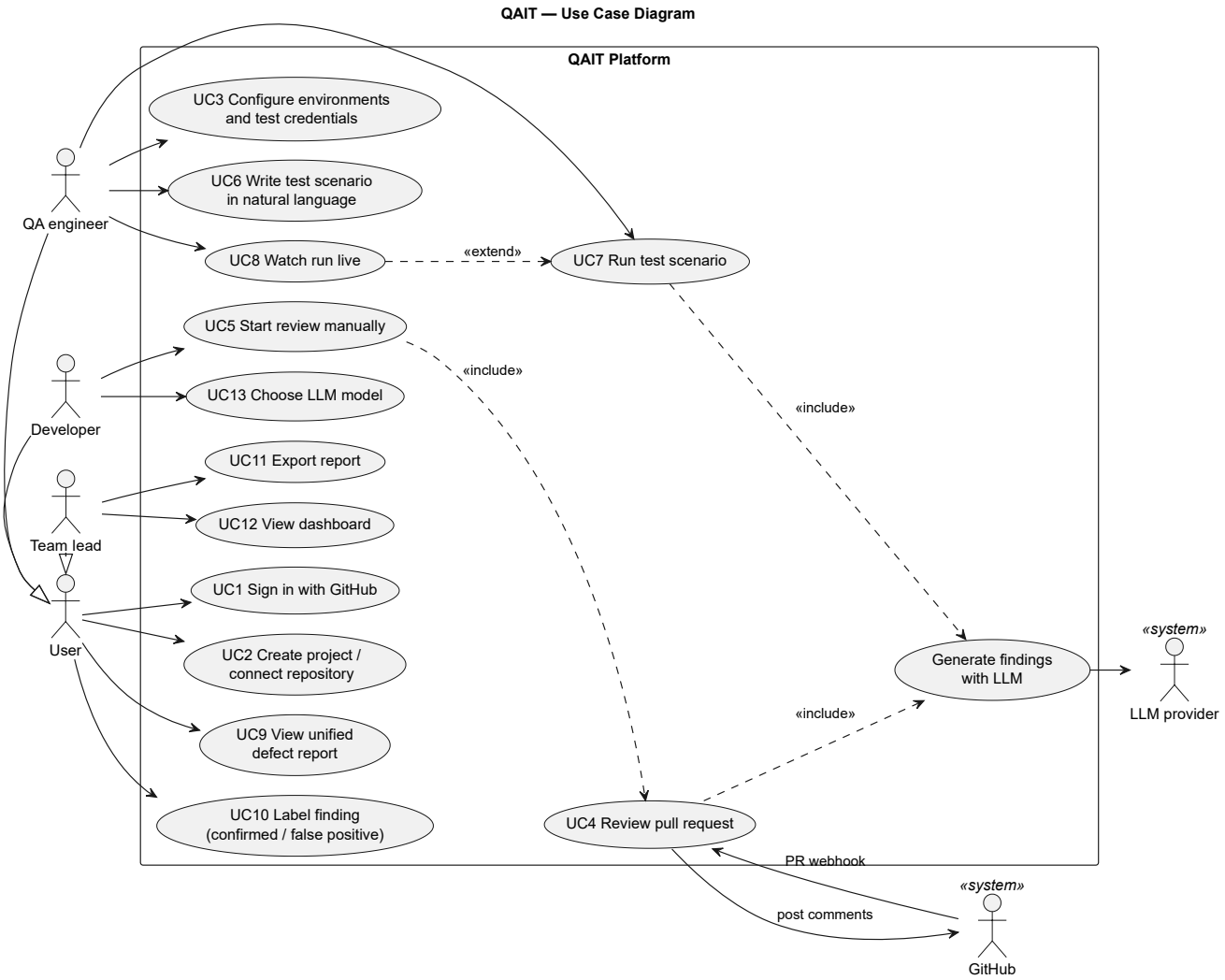
```
java -jar plantuml.jar -tsvg -o ../export diagrams/src/*.puml
```

7.1 System context



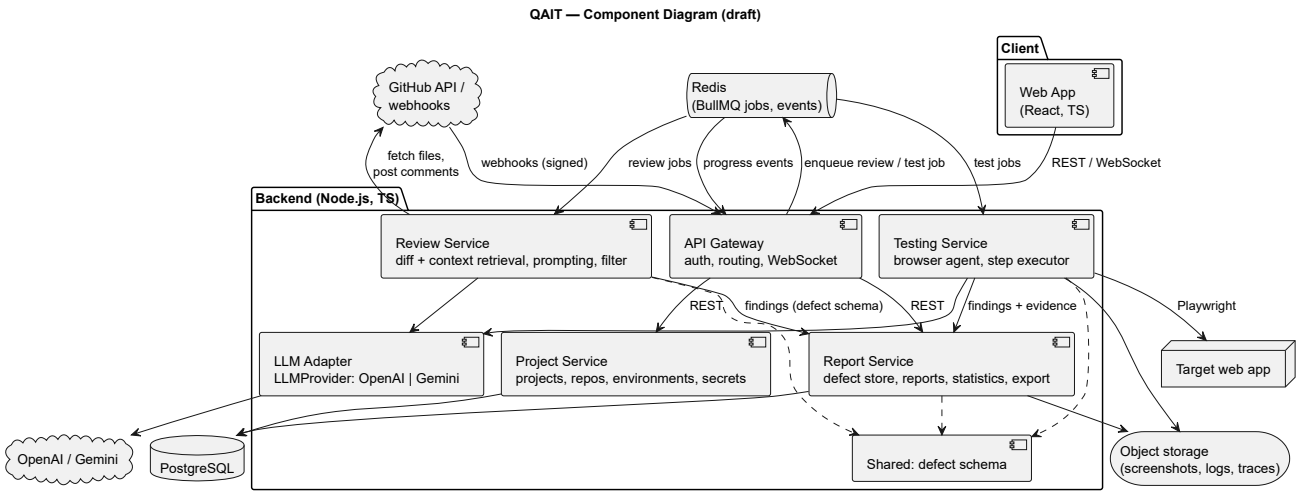
QAIT has three human user roles and three external systems: - **GitHub** is the source of code and pull-request events, and receives review comments. - **LLM providers** do the reasoning for review and testing. - **Target web application** is the system under test, driven through a real browser.

7.2 Use cases



Use case	Requirements
UC1 Sign in	FR-P1
UC2 Create project / connect repository	FR-P2
UC3 Environments and credentials	FR-P3, FR-P4
UC4 Review pull request	FR-CR1, CR3–CR6
UC5 Start review manually	FR-CR2
UC6 Write scenario	FR-T1
UC7 Run scenario	FR-T2, T3, T5
UC8 Watch run live	FR-T4
UC9 Unified report	FR-R2, FR-D1, D2
UC10 Label finding	FR-R3
UC11 Export	FR-R4
UC12 Dashboard	FR-R5
UC13 Choose model	FR-CR7

7.3 Components (draft)

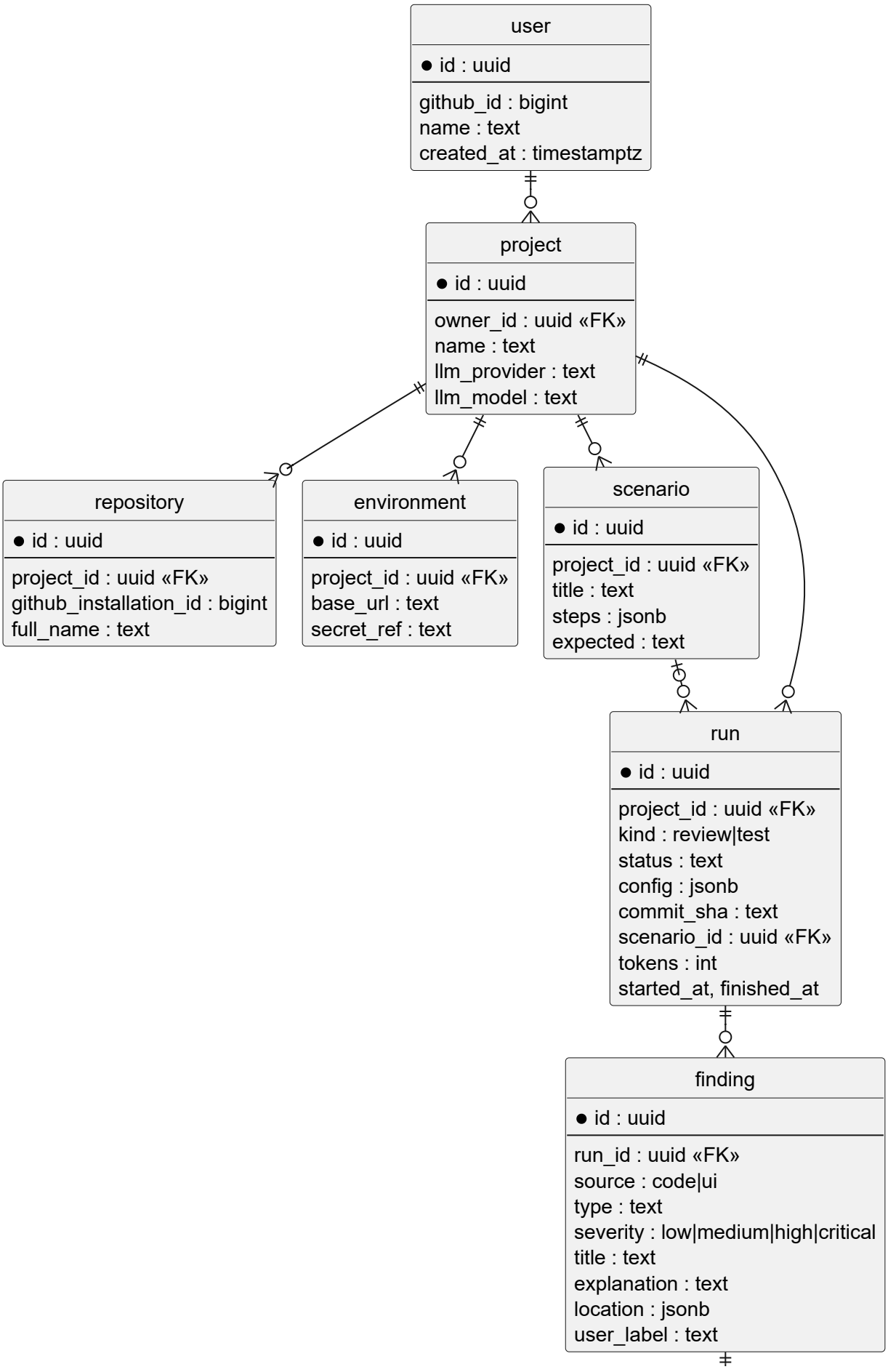


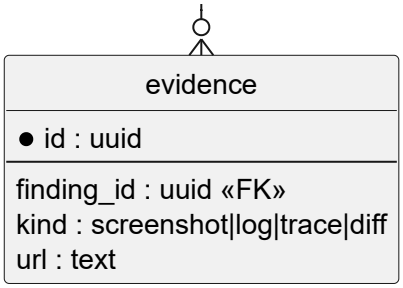
Component	Responsibility	Main requirements	Owner
Web App	UI for projects, scenarios, runs, report, dashboard	FR-P, *FR-R*, FR-T4	Нариман
API Gateway	Authentication (GitHub OAuth, JWT), routing, webhook verification, WebSocket fan-out	FR-P1, NFR-S2, NFR-S3	Бекнұр
Project Service	Projects, repositories, environments, encrypted secrets	FR-P2–P4, NFR-S1	Бекнұр
Review Service	Fetch diff and related files, build context, prompt, filter, post comments	FR-CR*	Бекнұр
Testing Service	Browser agent: page snapshot, action choice, execution, evidence capture	FR-T*	Бекнұр, Максот
Report Service	Store findings, unified report, labels, statistics, export	FR-D*, FR-R*	Бекнұр, Нариман
LLM Adapter	`LLMProvider` interface; OpenAI and Gemini adapters; token and cost accounting	FR-CR7, NFR-M2, NFR-C1	Бекнұр
Evaluation tools	Batch runs over the bench, CSV export, metrics	FR-EV*	Максот

Interaction. The client talks to the gateway over REST and WebSocket. Long-running work (reviews, test runs) is put on a Redis/BullMQ queue, so the gateway never waits on an LLM call. This allows retries (NFR-R1) and lets the review and testing workers be scaled separately. Workers publish progress events, and the gateway forwards them to the client.

7.4 Data model (draft)

QAIT — Data Model (draft)

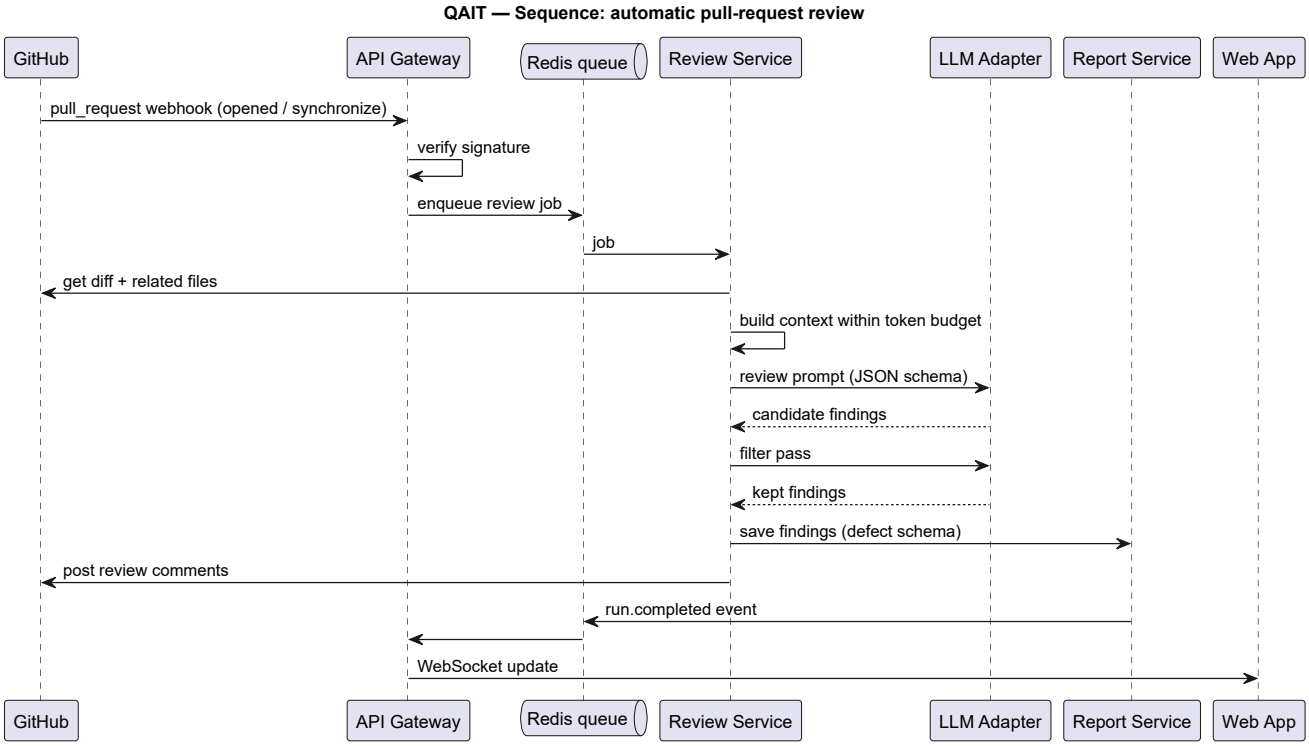




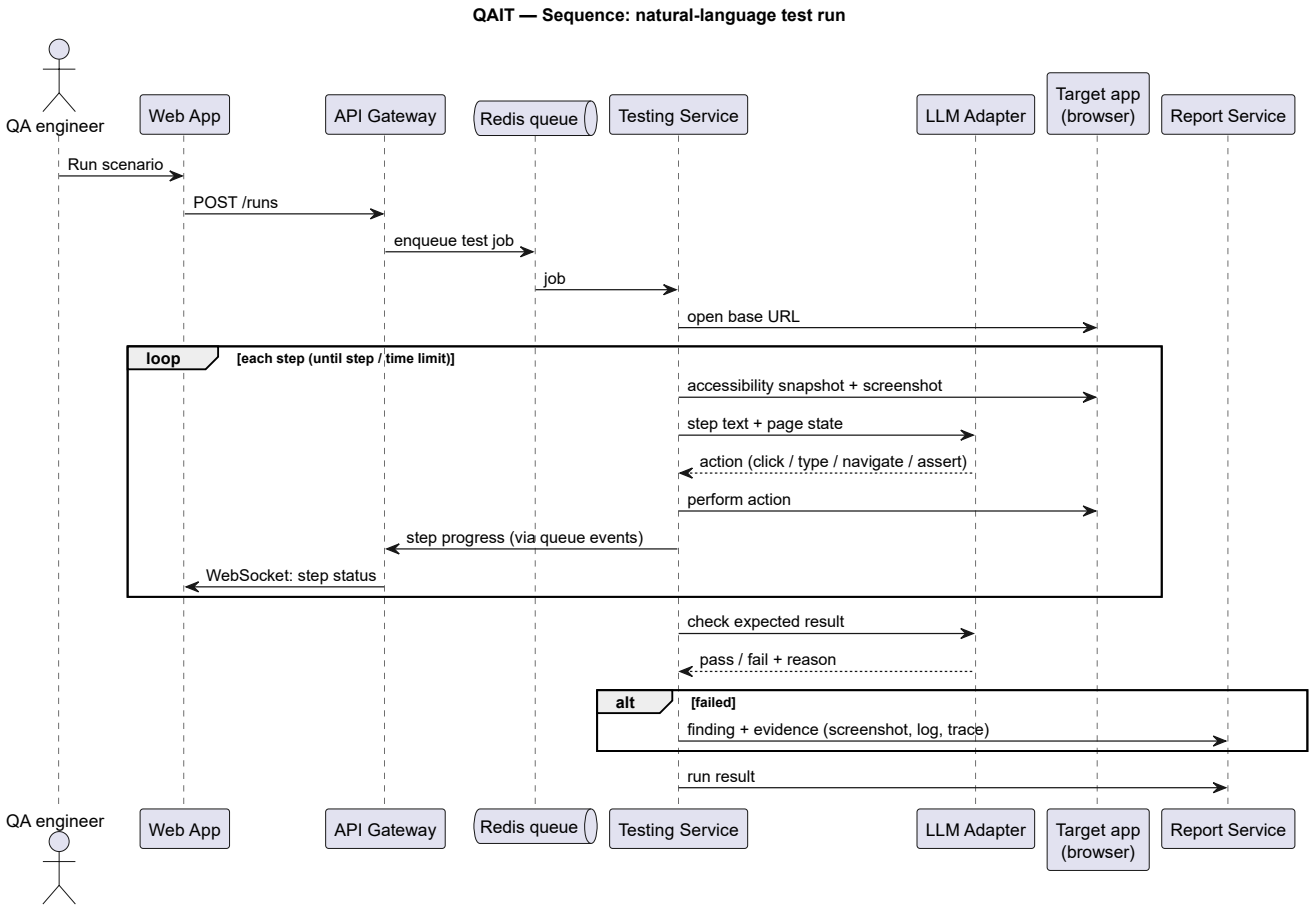
A **run** is one execution of either a review (`kind = review`, tied to a commit) or a test (`kind = test`, tied to a scenario). Both kinds produce **findings** in the same schema. The schema is in `src/shared/defect.schema.json`. The unified report (FR-R2) is a query over findings for a commit range or release, joined across both run kinds.

7.5 Key scenarios

Automatic PR review



Natural-language test run



7.6 Architectural decisions

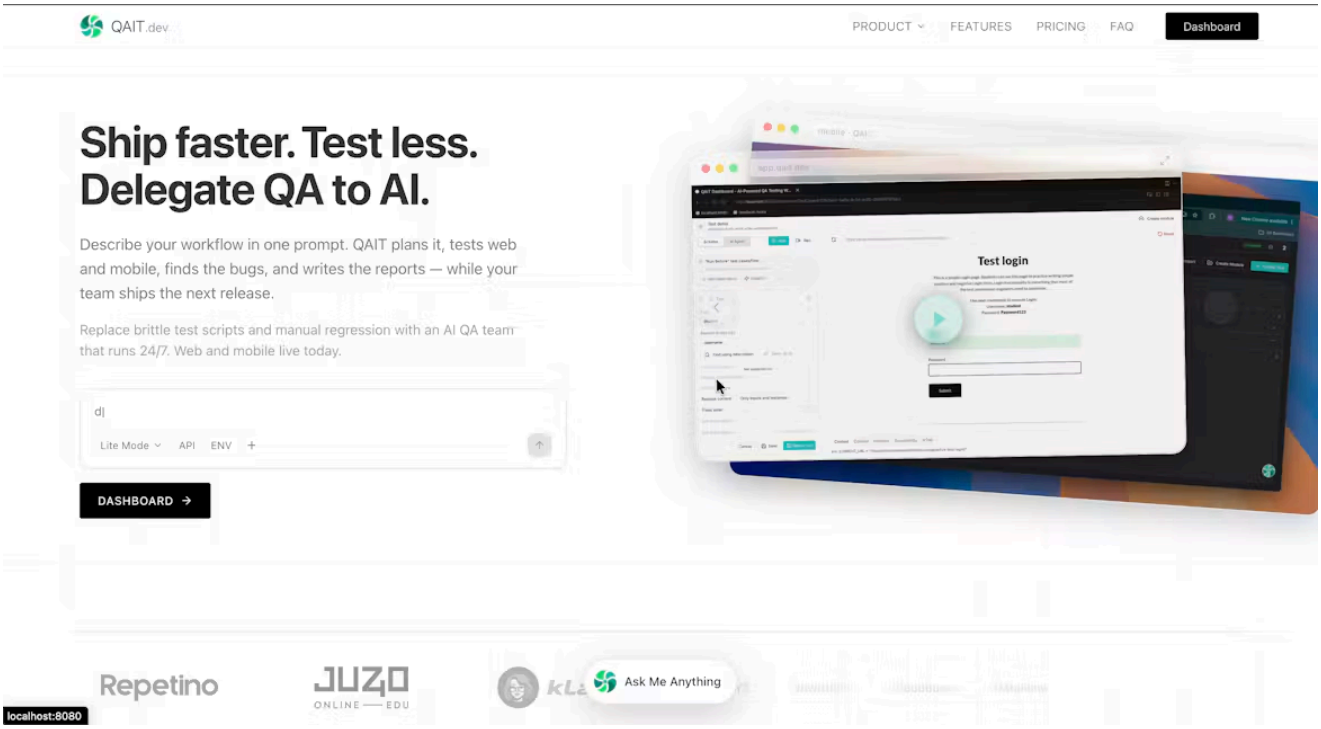
ID	Decision	Alternatives	Reason
AD-1	Separate review and testing workers behind a queue	One monolith	Very different run time and resources (browser vs. API calls); independent scaling and failure
AD-2	One defect schema shared by all services	Per-service formats	Required for the unified report and for the evaluation (same matching against ground truth)
AD-3	LLM behind an adapter interface	Direct SDK calls	Model comparison in experiments; vendor independence
AD-4	Page state for the agent = accessibility snapshot + screenshot, not full HTML	Full DOM; screenshot only	Fewer tokens and better element choice [L10]; screenshot kept as evidence
AD-5	Two-pass review (generate, then filter)	Single pass	Higher precision reported in industry [L7]
AD-6	Evidence files in object storage, metadata in PostgreSQL	Blobs in DB	Database size and backup cost

8. UX / prototype

Owner: Қайдар Н.Н.

Screens: `docs/prototype/`

Starting point: QAIT.dev



QAIT.dev is the team's prior product: a prompt-driven AI testing tool for web and mobile. Its visual style and its test editor are the starting point for the testing screens (S3). The code-review screens (S2), the unified report (S4) and the dashboard (S5) are new for this project; their mockups are below.

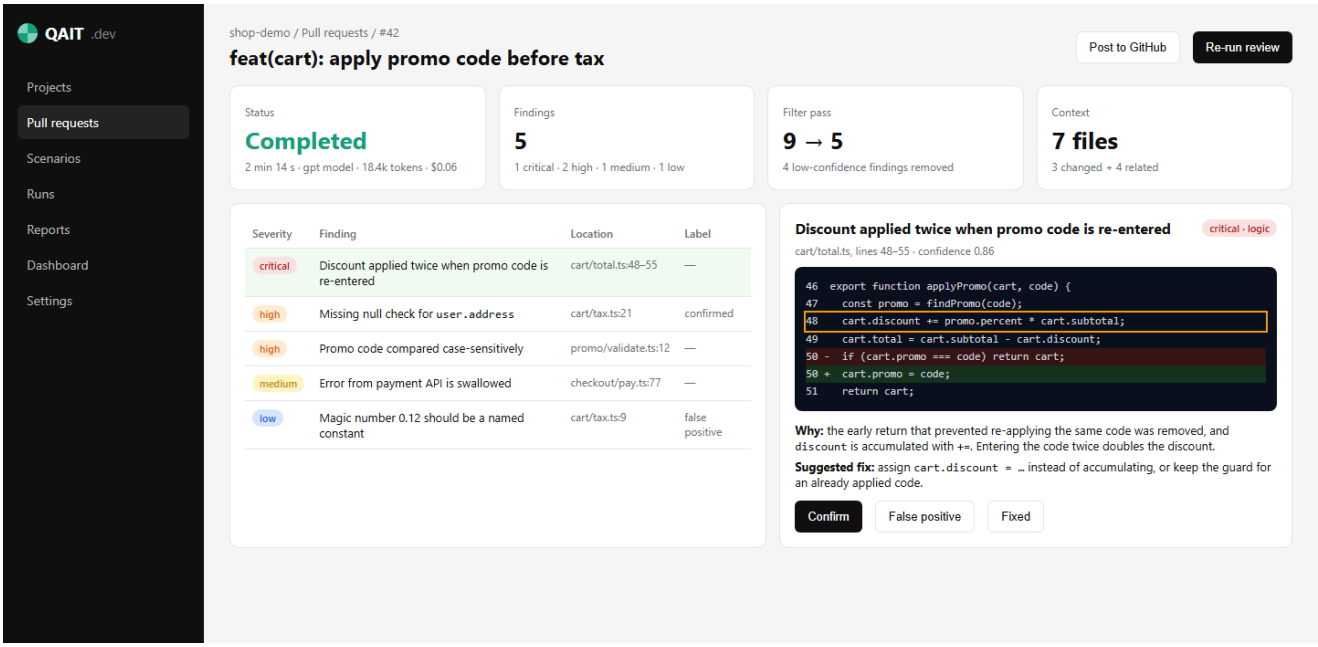
Key scenarios (planned screen flow)

#	Scenario	Screens	Use cases
S1	First start: sign in, create project, connect GitHub repo	Sign-in → Projects → New project → GitHub App install → Project overview	UC1, UC2
S2	Review a pull request	Project → Pull requests → Review run (progress) → Findings list → Finding detail (code + explanation + suggested fix)	UC4, UC5
S3	Write and run a test	Scenarios → Editor (steps, expected result, environment) → Live run (step list + screenshot) → Result	UC6, UC7, UC8
S4	Unified report	Reports → Release / commit range → Findings from code and UI, filters by source, severity, type → Label finding	UC9, UC10
S5	Dashboard	Findings by type / severity, precision from labels, trend	UC12

Mockups of key screens

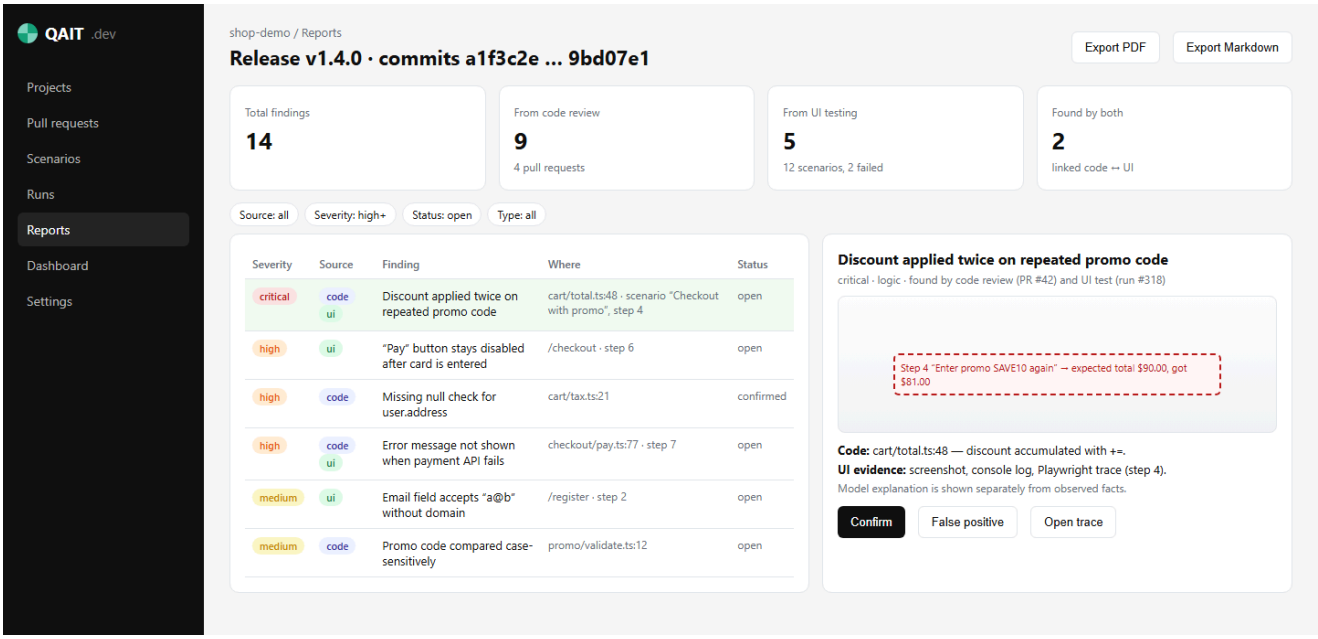
Static mockups (HTML source: `prototype/src/mockups.html`); data on the screens is illustrative.

S2: Pull-request review



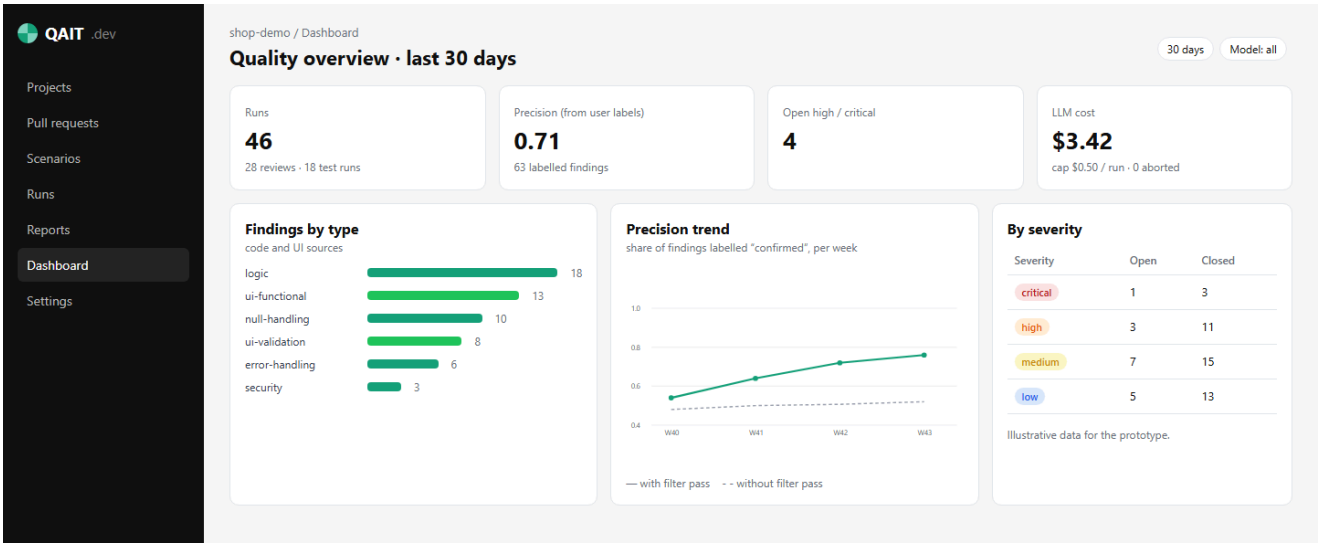
The run summary shows status, model, tokens and cost (NFR-C1) and the effect of the filter pass (FR-CR5). Findings are ranked by severity. The selected finding shows file and lines, the diff with the flagged line, the reason and a suggested fix (FR-CR4), and the user labels it as confirmed, false positive or fixed (FR-R3).

S4: Unified report



Findings from code review and UI testing for one release are shown together, with source, severity and status filters (FR-R2). A defect found by both is linked: code location, the failing scenario step and the UI evidence (screenshot, log, trace; FR-D2). Export to PDF or Markdown (FR-R4).

S5: Dashboard



Runs, precision from user labels, open high and critical defects, LLM cost against the per-run cap (FR-P6); findings by type, precision trend with and without the filter pass, defects by severity (FR-R5).

Example API (for external systems)

```
POST /api/projects/{projectId}/runs
Content-Type: application/json

{ "kind": "test", "scenarioId": "b6e1…", "environmentId": "9c0a…" }
```

```
{ "runId": "4f2d…", "status": "queued" }
```

`GET /api/runs/{runId}/findings` returns an array of findings in the defect schema.

9. Verification and evaluation plan

Owner: Сариев М.Б. The plan answers two questions: 1. **Verification:** does the platform meet its requirements? 2. **Evaluation:** how accurately does it find defects compared with a baseline? This is the research part.

9.1 Verification (does it work as specified)

Level	What	Tool	Requirements covered	When
Unit	Context builder, schema validation, provider adapters, report queries	Vitest / Jest	FR-CR3, FR-D1, NFR-M2	Every PR (CI)
API / integration	REST endpoints, auth, access rules, webhook signature	Supertest + test DB in Docker	FR-P*, NFR-S2, NFR-S3	Every PR (CI)
End-to-end	Main user flows in the web client	Playwright	UC1, UC2, UC7, UC9	Before each pre-defense
Reliability	Kill the worker during a job; LLM error injection	Scripted chaos test	NFR-R1	Pre-defense 2
Stability	10 repeated runs of each bench scenario on unchanged app	Batch runner	NFR-R2	Pre-defense 2 and final
Performance	Review time on bench PRs; API latency with 20 users	Batch runner, k6	NFR-P1, NFR-P2	Final
Usability	5 participants connect a repo and run the first review	Moderated test, SUS questionnaire	NFR-U1	Final
Security	Secret scanning, dependency audit	gitleaks, npm audit in CI	NFR-S1	Every PR (CI)

Every **M** requirement in 06-requirements.md is covered by at least one test. The traceability matrix is kept in `tests/traceability.csv` .

9.2 Evaluation (how well it finds defects)

Research questions

- **RQ1.** What precision, recall and F1 does LLM code review reach on seeded code defects, compared with static analysers?
- **RQ2.** What precision, recall and F1 does the browser agent reach on seeded UI defects, compared with scripted Playwright tests?
- **RQ3.** How much do code review and UI testing overlap, and how much does combining them add (union recall)?
- **RQ4.** How do model choice (e.g. GPT vs Gemini) and the filter pass affect accuracy, time and cost?

Test bench

Part	Content	Ground truth
B1 Code bench	≥ 10 small/medium open-source TypeScript / Python repositories. In each, ≥ 5 pull requests with **seeded** defects (mutation-style: off-by-one, missing null check, wrong condition, unhandled error, injection, race) plus clean PRs as controls	Defect list: file, line range, type
B2 Public data	Sample of CodeReviewer dataset [L2] comments marked as defect-related	Human review comments

Part	Content	Ground truth
B3 Web bench	≥ 3 web apps (e.g. a demo shop, a to-do app, an admin panel) with seeded UI defects: broken validation, wrong navigation, disabled button, wrong total, missing error message	Defect list: page, scenario, type
B4 Cross bench	≥ 2 apps where the same seeded defect is visible both in code and in UI	For RQ3

Target size: **≥ 100 seeded defects** in total. Defects are written by Максот and reviewed by a second team member. Members who tune prompts do not see the bench defects (prevents overfitting).

Matching rule

- **Code finding:** a true positive (TP) when it names the same file and its line range overlaps the seeded defect's range ±3 lines, and its type is compatible.
- **UI finding:** a TP when the same scenario fails at the step where the defect shows.
- Findings that match no seeded defect are reviewed manually and marked FP, or "real but unseeded". The second category is reported separately.

Metrics

- Precision = TP / (TP + FP), Recall = TP / (TP + FN), F1.
- Per defect type and per severity.
- Overlap (RQ3): defects found only by review, only by testing, by both; union recall.
- Cost and time: tokens, USD, wall-clock per run (RQ4).
- Stability: share of repeated runs with the same verdict.

Baselines

For	Baseline	Why
RQ1	Static analysers: ESLint + Semgrep (TS), Pylint + Semgrep (Python)	Common non-LLM practice [L14]
RQ2	Scripted Playwright tests written once for the original app version	Common practice; brittle under UI change [L11]
RQ2-robustness	Same scripted tests after controlled UI changes (renamed ids, moved elements) vs. agent	Measures maintenance benefit

Conditions

- Fixed model versions and temperature 0. Each configuration is run 3 times and reported as mean ± std.
- Same hardware / Docker image for all runs. Configuration ID stored with every result (FR-EV1).
- Statistical test: McNemar's test for paired detection on the same defects (platform vs baseline), $\alpha = 0.05$.

Success criteria (hypotheses)

ID	Criterion
H1	LLM review recall on B1 is higher than the static-analyser baseline
H2	LLM review precision ≥ 0.6 with the filter pass
H3	After controlled UI changes, agent recall drops less than scripted-test recall
H4	Union of review + testing has higher recall on B4 than either alone

These are hypotheses. The report will state whether each is supported or rejected.

9.3 Threats to validity

- **Seeded vs. real defects:** seeded defects may be easier. Mitigation: include B2 real review data and report it separately.
- **Data contamination:** models may have seen public repositories [L16]. Mitigation: seed new defects and use recent or private bench code.
- **Non-determinism:** mitigation by repeated runs and reporting variance.
- **Small bench:** report confidence intervals and avoid broad claims.

10. Project management and risks

10.1 Tools and workflow

- **Repositories:** back, worker, front. `main` is protected and changes come only by pull request with one approving review from another team member.
- **YouTrack:** https://qait-diploma.youtrack.cloud/projects/QAIT, project key `QAIT`. The backlog in 10.4 is imported from youtrack-backlog.csv. Each task has an assignee, expected outcome, acceptance criteria, deadline and priority.
- **Branch / commit convention:** `QAIT-12-defect-schema`, commit `QAIT-12: Add defect schema draft`. The PR description links the task and says how the change was checked, and the YouTrack task links back to the PR.
- **Tags:** `pre-defense-1`, `pre-defense-2`, `final-pre-defense`.
- **Meetings:** weekly with the supervisor, and a short team sync twice a week.

10.2 Responsibilities

Area	Responsible	Reviewer
Web client, UX prototype, report and dashboard	Қайдар Н.Н.	Таниберген Б.Б.
Gateway, services, database, GitHub integration, deployment	Таниберген Б.Б.	Сариев М.Б.
Test bench, evaluation, verification, experiments	Сариев М.Б.	Қайдар Н.Н.
Documentation sections	Each owner for their area	Supervisor

10.3 Milestones

Dates after M1 are tentative and will be aligned with the School's pre-defense schedule.

#	Milestone	Deliverables	Due
M1	Pre-defense 1	Docs 01–11, diagrams, prototype, YouTrack backlog	24 Oct 2026
M2	Skeleton	Monorepo, CI, Docker Compose, auth, project CRUD	15 Nov 2026
M3	Code review MVP	GitHub App, review service, findings in UI	13 Dec 2026
M4	Testing MVP	Browser agent, scenarios, evidence	31 Jan 2027
M5	Pre-defense 2	Unified report, first bench results	28 Feb 2027
M6	Evaluation	Full bench, baselines, statistics	4 Apr 2027
M7	Final pre-defense	Explanatory note, final results, demo	30 Apr 2027

10.4 Initial backlog (YouTrack)

Import: YouTrack → project → Import → CSV → youtrack-backlog.csv. Map *Assignee* to team logins and *Fix versions* to milestones M1–M7.

ID	Task	Assignee	Priority	Milestone
QAIT-1	Project proposal and scope	Бекнұр	High	M1
QAIT-2	Interview guide; conduct and summarise 3–5 interviews	Нариман	High	M1
QAIT-3	Literature review and analysis table	Максот	High	M1

ID	Task	Assignee	Priority	Milestone
QAIT-4	Existing system comparison, with evidence log	Нариман	High	M1
QAIT-5	Technology comparison	Бекнұр	High	M1
QAIT-6	Requirements and acceptance criteria	Максот	High	M1
QAIT-7	Context, use case, component diagrams	Бекнұр	High	M1
QAIT-8	UX prototype of key scenarios	Нариман	High	M1
QAIT-9	Verification and evaluation plan	Максот	High	M1
QAIT-10	Defect schema v1	Бекнұр	High	M2
QAIT-11	Monorepo, CI, Docker Compose	Бекнұр	High	M2
QAIT-12	GitHub OAuth sign-in	Бекнұр	High	M2
QAIT-13	Web client skeleton, routing, auth screens	Нариман	High	M2
QAIT-14	GitHub App + webhook handling	Бекнұр	High	M3
QAIT-15	Review service: context builder and prompt	Бекнұр	High	M3
QAIT-16	Run history and finding view	Нариман	High	M3
QAIT-17	Bench B1: select repos, seed defects	Максот	High	M3
QAIT-18	Testing service: agent loop with Playwright	Бекнұр	High	M4
QAIT-19	Scenario editor and live run view	Нариман	Medium	M4
QAIT-20	Bench B3: web apps with seeded defects	Максот	High	M4
QAIT-21	Unified report and export	Нариман	High	M5
QAIT-22	Batch runner and CSV export	Максот	High	M5
QAIT-23	Baselines: static analysers, scripted tests	Максот	High	M6
QAIT-24	Dashboard	Нариман	Medium	M6

10.5 Risks

#	Risk	Prob.	Impact	Mitigation	Owner
R1	LLM output unstable or not in schema	High	High	Structured output, schema validation with retry, temperature 0, repeated runs	Бекнұр
R2	LLM API cost exceeds budget	Medium	Medium	Token budget per run, cost logging, smaller models for the filter pass, cached bench runs	Бекнұр
R3	Browser agent fails on complex UIs	High	Medium	Step and time limits, simplified page state, bench apps of controlled complexity	Максот
R4	Bench too small or biased for conclusions	Medium	High	≥ 100 defects, second reviewer, real-data subset B2	Максот
R5	API provider changes or rate limits	Medium	Medium	Adapter interface, second provider	Бекнұр
R6	Workload imbalance, a member falls behind	Medium	High	Weekly review of YouTrack, re-plan tasks, cross-review	All
R7	GitHub App permissions / webhook issues in demo	Low	High	Test repo for demo, recorded fallback video	Бекнұр
R8	Data privacy: sending customer code to LLM	Medium	Medium	Bench uses open or own code; self-hosted option documented	All
R9	Agent "passes" a test by acting on the wrong element, hiding a real bug	Medium	High	Log every element choice with reason and confidence; confidence threshold; seeded defects measure false passes	Максот

#	Risk	Prob.	Impact	Mitigation	Owner
R10	Test credentials leak into prompts or logs	Low	High	Credentials stored as secrets and injected at action time, never sent to the LLM; log redaction	Бекнұр
R11	Scope too large for the diploma	Medium	High	Focus evaluation on two ideas: review + testing complementarity and robustness to UI change	All

10.6 Dependencies

- OpenAI and Gemini API keys (team budget).
- GitHub organisation for the GitHub App and test repositories.
- Hosting for the demo (VPS or cloud credits).
- Supervisor review at each milestone.

11. Supervisor feedback

Supervisor: Айтмұханбетова Э.А.

#	Date	Deliverable	Comment (supervisor)	Decision / action	Task	Status
1	24 Oct 2026	Pre-defense 1 materials (repository, documentation)	Oral review: the materials are in good order; no specific changes requested	Continue with the plan for milestone M2	QAIT-1	Closed
2						
3						

Entry 1 is the team's summary of an oral discussion with the supervisor. Later comments are recorded here with the commit or pull request that addresses them.

Appendix A. Repository README

Diploma project, Astana IT University, School of Software Engineering, 2026–2027.

QAIT is a web platform that combines **LLM-based code review of Git repositories** and **AI-driven testing of web applications by a browser agent**, and merges the results of both into **one defect report**. The research part measures how accurately each approach finds defects and how far the two complement each other.

Problem statement

Manual code review and manual / scripted testing take a large share of development time. Scripted UI tests (Selenium, Playwright, Cypress) break when the interface changes and need constant maintenance. Static analysers find only part of the defects and produce many false positives. Small teams without a dedicated QA-automation engineer often skip both.

Large language models can read code changes and drive a browser from a natural-language description, but existing tools do either code review **or** testing, keep the results in separate places, and rarely report how accurate they are.

Target users

- Small and medium development teams without a dedicated test-automation engineer.
- QA engineers who want to describe test scenarios in natural language instead of scripts.
- Team leads who need one view of the defects found in a release.

Aim

Design, build and evaluate a platform that automates code review and web testing with LLMs and presents their findings in a single defect report.

Objectives

1. Analyse the users' code-review and testing workflows and derive requirements (see docs/02-user-research.md).
2. Review the literature on LLM-based code review, LLM-based testing and web agents.
3. Compare existing systems and technology alternatives, and justify the choices.
4. Design a service-based architecture with a common defect format.
5. Implement the code-review service (GitHub integration, pull-request analysis).
6. Implement the testing service (browser agent that runs natural-language scenarios).
7. Implement the web client with the unified report and dashboard.
8. Build a test bench of repositories and web applications with known (seeded) defects and evaluate precision, recall and F1 against a baseline.

Expected outcomes

- A working web platform with code-review and testing services and one defect report.
- A test bench with seeded defects and a reproducible evaluation method.
- An experimental comparison of LLM configurations against a baseline (static analysers / scripted tests).

Scope

In scope: GitHub repositories (public and private through a GitHub App), pull-request review, web applications reachable by URL, natural-language test scenarios, OpenAI and Gemini models, the unified report, the evaluation bench.

Out of scope: GitLab / Bitbucket, native mobile apps, load and security testing, self-hosted model training.

Full scope: docs/01-proposal.md.

Team

Student	Group	Responsibility
Қайдар Нариман Нұрланұлы	SE-2406	Web client, unified report, dashboard, UX prototype
Танибергen Бекнұр Болатбекұлы	SE-2406	Backend, service architecture, GitHub integration, database, deployment
Сариев Максот Бахтыгалиевич	SE-2406	Test bench, evaluation method, experiments, platform testing

Supervisor: Айтмұханбетова Эльвира Айтмұханбетқызы, senior lecturer, School of Software Engineering.

Relation to prior work

Team members previously worked on QAIT.dev, an AI platform for automated web and mobile testing. The comparison with QAIT.dev is part of docs/04-existing-systems.md.

Current status

Stage: **pre-defense 1** (analysis, design, planning).

Area	Status
Proposal, scope	Done
User research	In progress
Literature review, comparisons	Done (first version)
Requirements	Done (first version)
Architecture, diagrams	Done (draft)
UX prototype	In progress
Verification plan	Done (first version)
Implementation	

Technologies

Layer	Choice	Justification
Web client	React, TypeScript, Vite	docs/05
Backend services	Node.js (TypeScript, Express)	docs/05
Database	PostgreSQL	docs/05
Service interaction	REST + Redis-based job queue (BullMQ), WebSocket for live status	docs/05
Browser automation	Playwright	docs/05
LLM providers	OpenAI, Google Gemini (pluggable)	docs/05

Layer	Choice	Justification
Deployment	Docker, GitHub Actions	

Links

- YouTrack: https://qait-diploma.youtrack.cloud/projects/QAIT (import file: docs/youtrack-backlog.csv)
- Source code repositories:
- Backend: https://github.com/bekhop2006/qait-dimploma-back
- Worker (testing runner): https://github.com/bekhop2006/qait-dimploma-worker
- Web client: https://github.com/bekhop2006/qait-dimploma-front
- Prototype: docs/08-ux-prototype.md
- Documentation index:

#	Document
1	Project proposal and scope
2	User research (evidence of relevance)
3	Literature review
4	Existing system comparison
5	Technology comparison
6	Requirements specification
7	Architecture and diagrams
8	UX prototype
9	Verification and evaluation plan
10	Project management and risks
11	Supervisor feedback
—	Pre-defense 1 submission report
—	Requirements checklist

Repository structure

```
src/           application code, one folder per component (web, gateway, review-service, testing-
service, report-service)
docs/          proposal, research, reviews, comparisons, requirements, decisions
diagrams/      editable sources (.puml) and exports (.svg / .png)
tests/         automated tests and the evaluation bench
config/        configuration examples (.env.example)
```

Setup, run and test

No runnable code is submitted at this stage. Instructions will be added with the first service.

```
cp config/.env.example .env
# docker compose up   (from pre-defense 2)
```

Workflow

- `main` is the reviewed baseline; work is done in branches named after the YouTrack task, e.g. `QAIT-12-defect-schema`.
- Commit messages start with the task ID: `QAIT-12: Add defect schema draft`.
- Every change goes through a pull request reviewed by another team member.
- Each pre-defense submission is tagged: `pre-defense-1`, `pre-defense-2`, `final-pre-defense`.

Appendix B. YouTrack backlog

Project: https://qait-diploma.youtrack.cloud/projects/QAIT. Acceptance criteria of each task are in its description in YouTrack.

Issue Id	Summary	Assignee	Priority	State	Due Date	Fix versions
QAIT-1	Project proposal and scope	Таниберген Бекнұр	Major	In Progress	2026-10-24	M1
QAIT-2	Interview guide; conduct and summarise 3–5 interviews	Қайдар Нариман	Major	In Progress	2026-10-24	M1
QAIT-3	Literature review and analysis table	Сариев Максот	Major	In Progress	2026-10-24	M1
QAIT-4	Existing system comparison with evidence log	Қайдар Нариман	Major	In Progress	2026-10-24	M1
QAIT-5	Technology comparison	Таниберген Бекнұр	Major	In Progress	2026-10-24	M1
QAIT-6	Requirements and acceptance criteria	Сариев Максот	Major	In Progress	2026-10-24	M1
QAIT-7	Context, use case and component diagrams	Таниберген Бекнұр	Major	In Progress	2026-10-24	M1
QAIT-8	UX prototype of key scenarios	Қайдар Нариман	Major	In Progress	2026-10-24	M1
QAIT-9	Verification and evaluation plan	Сариев Максот	Major	In Progress	2026-10-24	M1
QAIT-10	Defect schema v1	Таниберген Бекнұр	Major	Open	2026-11-15	M2
QAIT-11	Monorepo, CI and Docker Compose	Таниберген Бекнұр	Major	Open	2026-11-15	M2
QAIT-12	GitHub OAuth sign-in	Таниберген Бекнұр	Major	Open	2026-11-15	M2
QAIT-13	Web client skeleton, routing, auth screens	Қайдар Нариман	Major	Open	2026-11-15	M2
QAIT-14	GitHub App and webhook handling	Таниберген Бекнұр	Major	Open	2026-12-13	M3
QAIT-15	Review service: context builder and prompt	Таниберген Бекнұр	Major	Open	2026-12-13	M3
QAIT-16	Run history and finding view	Қайдар Нариман	Major	Open	2026-12-13	M3
QAIT-17	Bench B1: select repositories and seed defects	Сариев Максот	Major	Open	2026-12-13	M3
QAIT-18	Testing service: agent loop with Playwright	Таниберген Бекнұр	Major	Open	2027-01-31	M4
QAIT-19	Scenario editor and live run view	Қайдар Нариман	Normal	Open	2027-01-31	M4
QAIT-20	Bench B3: web apps with seeded UI defects	Сариев Максот	Major	Open	2027-01-31	M4

QAIT-21	Unified report and export	Қайдар Нариман	Major	Open	2027-02-28	M5
QAIT-22	Batch runner and CSV export	Сариев Максот	Major	Open	2027-02-28	M5
QAIT-23	Baselines: static analysers and scripted tests	Сариев Максот	Major	Open	2027-04-04	M6
QAIT-24	Dashboard	Қайдар Нариман	Normal	Open	2027-04-04	M6